Inverse Kinematics on Arduino: 6-DoF Robotic Arm

Project Introduction - Markku Kirjava

Made: 05/2025

Source code: [GitHub - Inverse Kinematics on Arduino: 6-DoF Robotic Arm]
Platform: Arduino Uno + C++

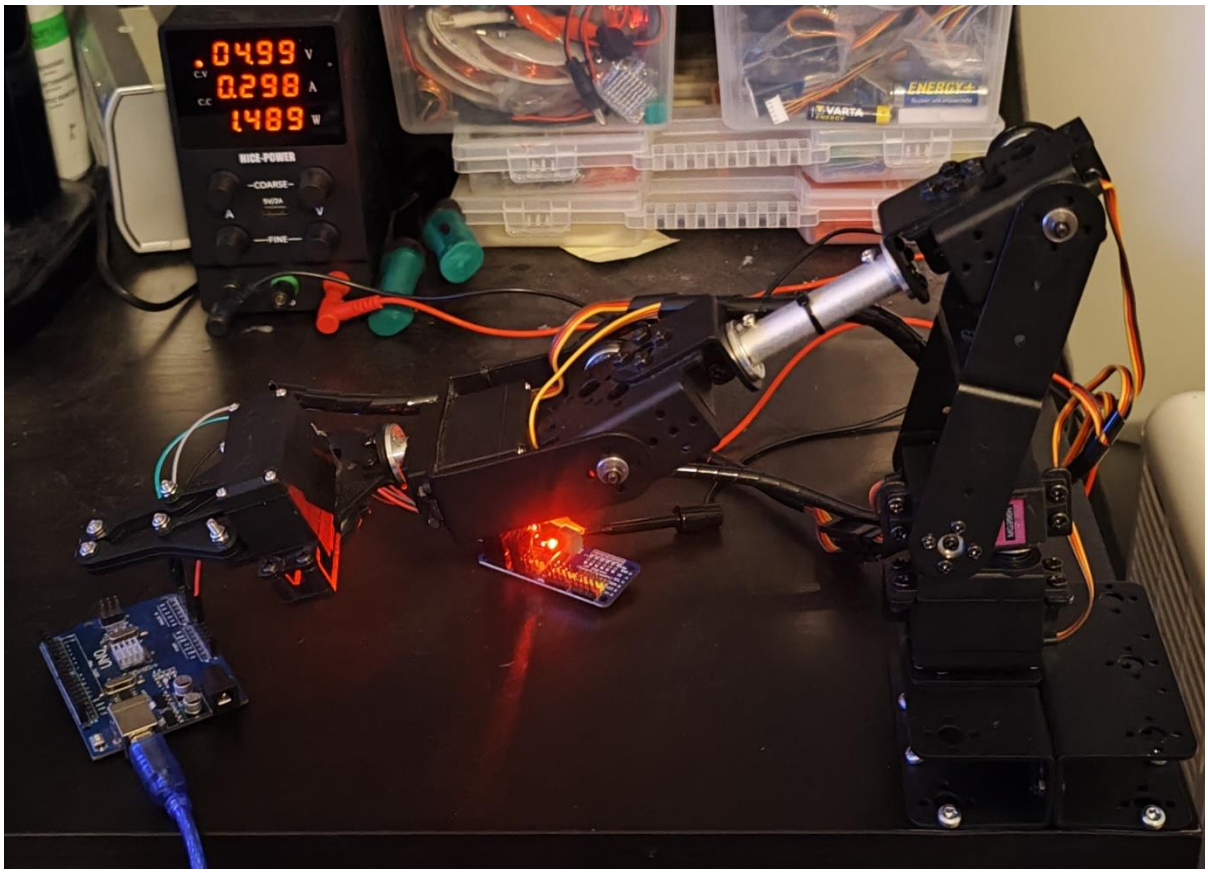

Table of Contents

Summary

This project introduction presents the development of an Arduino-based, six-degree-of-freedom (6-DoF) robotic hand from start to finish. The project was implemented in stages, with each stage focusing on building, testing and improving a specific area. The robotic hand operates on the principle of inverse kinematics (IK) calculation: the user provides a target coordinate in three-dimensional space, and the software calculates the corresponding joint angles, which are used by the servo motors to position the hand in the desired position.

A clear and modular software architecture was implemented during the project, which enables expansion and maintenance of the system. The servo motors were controlled safely using a separate PWM circuit, and the calculations considered the physical dimensions of the structure and the angular limitations of the servos. The movements of the robotic hand are controlled by commands entered via the serial port, which support both the movement of the hand and the control of the gripper.

This introduction goes through the design, implementation and development stages of the project in detail. Each phase describes the problems solved, decisions made, and results achieved. The summary also provides a comprehensive picture of the project's challenges and how they were addressed with technical and programmatic solutions.

Technical description

- Source code: GitHub - Inverse Kinematics on Arduino: 6-DoF Robotic Arm
- Development board: Arduino Uno.
- Servos: 6 × MG996R.
- PWM-controller: PCA9685 16-channel I2C servo-control circuit.
- Operating voltage: 5V from an external power supply for servos, 5V USB for Arduino.
- Programming language: C++ (Arduino IDE).
- Software architecture:
    - ik_solver.cpp/h: Inverse kinematics calculation (2D/3D).
    - servo_driver.cpp/h: PWM control.
    - interface.cpp/h: Serial port command handling.
    - Inverse_Kinematics_6-DoF_Arm.ino: Startup logic and system integration.
- Data communication: USB-serial (input commands from a PC).
- Component mechanics: 6-DoF robotic arm frame made of aluminium components.
- User inputs: Commands such as GOTO X=100 Y=80 Z=50 and GRIP CLOSE.

Project planning and scoping

The project began with a desire to understand how a robotic arm could be controlled programmatically so that it could follow three-dimensional movement commands. The goal was not to immediately build a fully-fledged six-degree-of-freedom (6-DoF) system, but to proceed step by step so that each development step brings a new learning experience and concrete functionality to the hardware.

It was important to clearly define the project right from the start: instead of trying to control all 6-DoF (six-degrees of freedom) at once, the focus was initially on the operation of the shoulder joints and the calculation of simple 2D inverse kinematics. This allowed for a reliable and understandable foundation, which was later expanded by adding new joint angles and expanding the controllable space to three dimensions.

The project objectives were written down as follows:

- Build a physical 6-DoF (six-degree-of-freedom) robotic hand on the Arduino platform using commonly available components.
- Implement servo motor control via a separate PWM circuit to achieve precise and simultaneous movement.
- Design software that can receive coordinates and guiding the robotic arm towards a target point.
- Use the project as a learning platform in the subject areas of kinematics and robotics and to deepen knowledge and understanding in servo mechanics and embedded software development.

Emphasizing practicality, the project's progress and expansion were planned to be implemented in phases, ensuring that something concrete to test and work on was achieved at each stage. This reduced risks and enabled better error management along the way.

Basic structure and component selection

The technical implementation of the project began with the assembly of the physical structure and the selection of the necessary components. Since the goal was to build a robotic arm that allows for multiple joint movements, components were chosen that are widely used in hobbyist circles and thus provide sufficient publicly available examples and modelling.

Selected components

- **6x MG996R-servos** – These metal gear servos were chosen for their affordability and sufficient torque, making them well suited for light to medium robotic applications.
- **PCA9685 PWM-circuit** – 16-channel PWM circuit that allows up to 16 servos to be controlled over a single I2C bus. This allows servos to be controlled precisely without running out of PWM channels on the Arduino.
- **Arduino Uno** – A suitable development board for the project, with sufficient I/O pins and support for the I2C bus. The Uno offers sufficient performance for inverse kinematics calculations and serial communication management.
- **External 5V power supply** – A separate power supply was needed to operate the servo motors, as the Arduino is not capable of supplying them with enough power safely.
- **6-DoF robotic arm** – Completed aluminium robot arm frame, to which the servos were attached in the planned order.

Structure and connections

The robotic arm was mechanically constructed so that they form the following structure by joints: base rotation, upper arm tilt, forearm, wrist tilt, wrist rotation and pincers. The PWM circuit was connected to the Arduino as follows (Figure 1.):

- **GND → Arduino GND**
- **VCC → Arduino 5V**
- **SDA → Arduino A4**
- **SCL → Arduino A5**

The channels of the PWM circuit were designed specifically for servos, and each channel had a signal connection, a power connector, and a ground connector. This meant that servos could be installed directly into the circuit without any additional wiring. In addition, a separate power supply could be connected directly to the PWM circuit, from which a separate power supply could be distributed directly to each servo.

At this stage, the project produced a working robotic arm structure whose servos could be controlled via code. This laid the foundation for the software architecture to be implemented in the next stage.

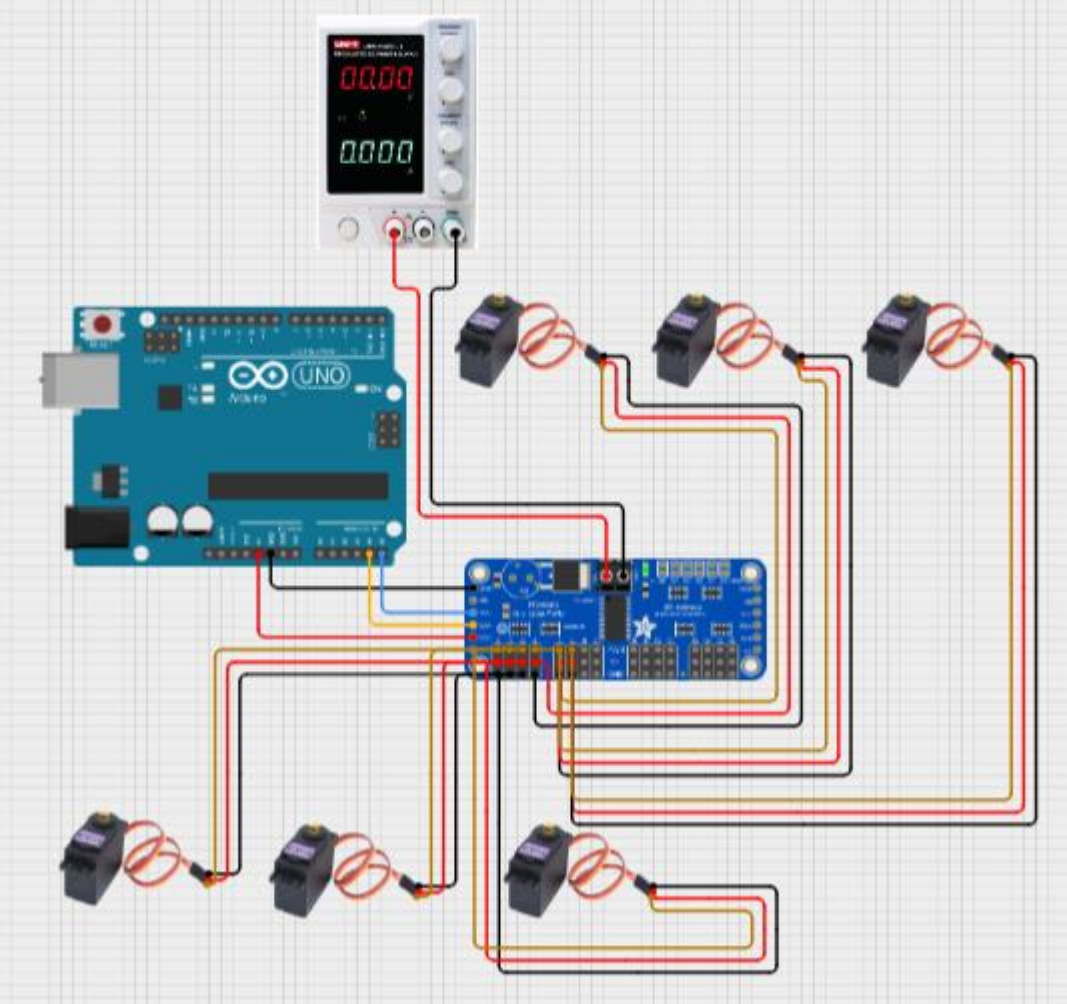

Figure 1. Circuit diagram.

Software architecture design and implementation

Once the physical structure of the robotic arm was complete, the next stage on project was move to software development planning. Since the goal of the project was to create an extensible, modular, and clear code set, the solution was implemented using C++ classes, with different areas of responsibility divided into their own modules.

main modules of the architecture

- **ik_solver.***
  This module is responsible for calculating inverse kinematics. Based on the target coordinates, it calculates the angles at which each joint must be positioned so that the head of the robot arm's gripper hits the correct spot.
- **servo_driver.***
  This is responsible for controlling the individual servos. It handles the conversion of angles to PWM values and communicates with the PCA9685 circuit via I2C.
- **interface.***
  This component reads commands coming through the serial port (e.g. GOTO X=100 Y=50 Z=75, GRIP OPEN) and parses them into internal commands that control the robot arm.
- **Inverse_Kinematics_Arduino_6DoF.ino**
  The actual Arduino main program that initializes the system, receives commands, and periodically updates the servo states.

Goals for software architecture

- **Modularity:** Each part of the program works as independently as possible, which makes it easier to find errors and add new functions.
- **Clarity and maintainability:** Class names and responsibilities have been kept as logical as possible, and the code structure follows a consistent model.
- **Extensibility:** In the future, the program can be expanded to include features such as trajectory interpolation, obstacle detection, or sensor utilization without having to completely rewrite existing components.

This stage laid a solid foundation for subsequent software development. The modular structure not only clarified project management but also enabled efficient testing and development in stages. Each part could be developed and tested independently without having to change the entire system. This approach proved to be very useful in the later stages of the project, when the system was expanded to include more DoF (degrees of freedom) and new functionalities were added. The careful design of the architecture laid the foundation for a robust and extensible software that is easy to further develop for future needs.

Servo control and execution of movement commands

At this stage of the project, the focus was on implementing the first practical functions of the system: receiving commands and controlling the servos. The goal was to get the joints of the robotic arm to move to predefined positions, and to ensure that the control worked reliably, and the movement was smooth and safe.

Servo control via PCA9685 circuit

The servo control was based on a PCA9685 circuit that communicated with the Arduino Uno over the I2C bus. This allowed for precise and simultaneous control of up to 16 servos. The implementation was a servo_driver module where each servo was defined as its own object and the control was implemented by converting the angle (in degrees) to a PWM signal per channel.

It became clear quite early on that sudden and rapid movements of the servos caused considerable mechanical stress on the robotic arm and posed a potential hazard to the environment. To make the movements more controlled and safer, it was decided to implement a soft motion mechanism for the servo control, which slowed down the movements and moved the arm to a new position gradually. However, this did not succeed overnight, but required several experiments and careful consideration of how the movement could be phased without jerking and delay. The process provided a wealth of lessons on, among other things, timing, servo response times and PWM signal management. In the end, the solution improved both the device's service life and overall safety and served as an excellent example of practical problem solving in robotics.

Motion commands and serial protocol

A separate interface module was developed to control the robotic arm, which receives and interprets text commands via a serial port. This solution allowed the robotic arm to be controlled directly from the computer using easily understandable commands. For example, the following commands enable movement and function control:

GOTO X=120 Y=50 Z=75

GRIP OPEN

GRIP CLOSE

The following were implemented in command processing:

- A verbal parser that interprets the command type (e.g. GOTO, GRIP) and its parameters.
- Simple error checking that skips invalid commands and does not attempt to execute them.
- Real-time control, where new commands can be issued while the system is running.

Movement route

At this stage, the system did not yet use actual inverse kinematics calculations but rather controlled the servos by directly giving them predefined angle values. This approach allowed testing the system design and verifying the servos' operation without complex calculations.

Motion commands received via the serial port were directly converted into individual servo angles, allowing the robotic arm to be controlled by simple manual commands. This provided a fast and clear way to validate the entire control chain, from command interpretation to servo movements.

Although the movements were not yet coordinate-based or dynamically calculated, this phase laid a solid foundation for later kinematic calculations and verified the functionality of the servos, power supply, and software control logic. It also provided an opportunity to examine the movements of the robotic arm in practice and identify early improvement needs, such as smoothness of movements and physical safety.

Simple 2D inverse kinematics calculation with 2-DoF (two-degrees of freedom)

At this stage, the first working version of the inverse kinematics solution was implemented, which allowed the robot arm to be oriented to a specific point in two-dimensional space. This stage formed the mathematical basis for the motion control of the entire system. For this stage, a real coordinate system was built under the robot arm, where the shoulder rotate servo of the robot arm stand was placed at the origin of the coordinate system (Figure 2.). Creating the real X,Y coordinate system was important and facilitated later practical testing and analysis of the robot arm's accuracy.

However, defining the coordinate system and adapting it to the physical world proved to be surprisingly challenging. A particular problem was ensuring that the computed coordinate system accurately corresponded to the actual physical location – even a small inaccuracy could lead to significant errors in the result of the movement. This offered a very new kind of learning alongside software development: coordinating geometry, measurement and mechanics required patience and repeated testing. The challenges forced to approach the problem from the perspective of a practical design engineer and created a good bridge between pure code and the physical world.

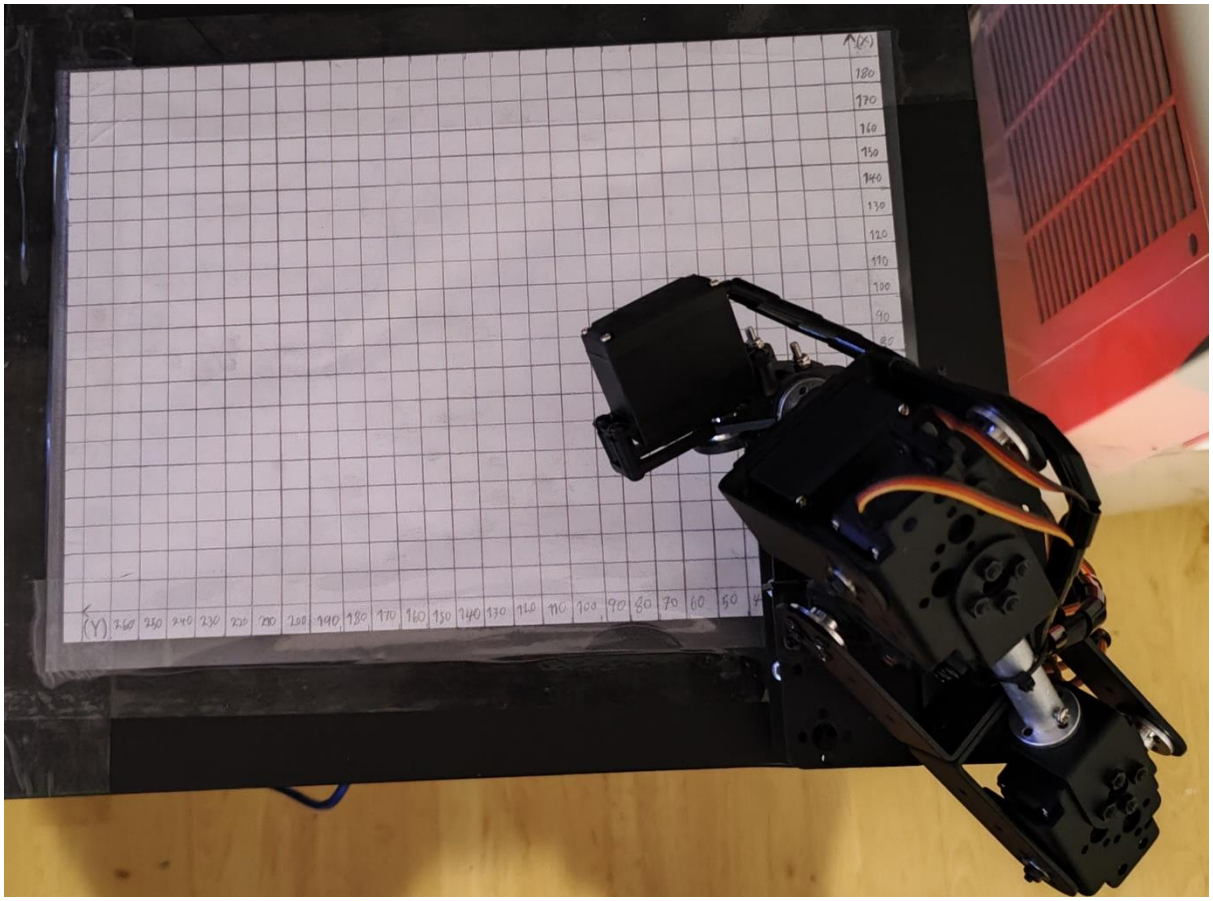

Figure 2. X,Y coordinate with the origin set directly below the shoulder rotate servo.

The goal is to move the hand to the desired (X, Y) coordinate point

At the begin, worked in a situation where the Z-axis was not yet considered, but the entire movement took place in the X,Y coordinate system. The following two servos were used:

- ***Shoulder rotate*:** rotates the arm horizontally (rotates the entire robot arm).
- ***Shoulder tilt*:** raises and lowers the arm (vertical tilt of the upper arm).

The goal was to find out which joint angles produce movement so that the end of the upper arm ("elbow") ends up at a specific point in the plane.

Trigonometric solution

The calculation was based on a simple two-link model. When known:

- the height of the stand (L1),
- the length of the arm (L3),
- and the X and Y coordinates of the target point,

the inverse kinematics can be solved as follows:

1. Apply the sine and cosine functions to find the angles.
2. Convert the calculated angles to servo control degrees.

Programmatically the C language Math library, which includes trigonometric functions, was used. At this point, it became clear that it was easier to programmatically implement the angle values in radians and convert them to degrees when communicating with the user (user input and program feedback).

Calibration and limitations

The first version introduced the following technical solutions:

- Fixed, roughly measured angle limits were set for the servos to prevent damage to the hardware.
- Serial port output to verify the value of the joint angles.
- Restriction to positive X and Y ranges, which simplified the initial testing phase.

This phase provided the first concrete experience of how a mathematical model produces real movement. It was an important milestone towards three-dimensional control and showed that the basic structure of the system was functional and worthy of further development.

3D extension and introduction of the Z dimension: 3-DoF (third degree of freedom)

Once the 2D inverse kinematics had been reliably implemented, the next step was to extend the robotic arm's computation to 3D space. The goal of this phase was to enable it to reach any point in a three-dimensional coordinate system using three servos.

Enabling the third servo

Implementing 3D computing required the inclusion of a new joint, the elbow servo, in the movement (Figure 3.). The first three servos built the following entity:

- ***Shoulder rotate* -servo** – rotates the entire arm horizontally (in the XY plane).
- ***Shoulder tilt* -servo** – raises the upper arm to the desired angle vertically.
- ***Elbow tilt* -servo** – tilt the position of the forearm as an extension of the upper arm.

Together, these three servos enabled the arm to reach any three-dimensional point within the range allowed by the kinematics.

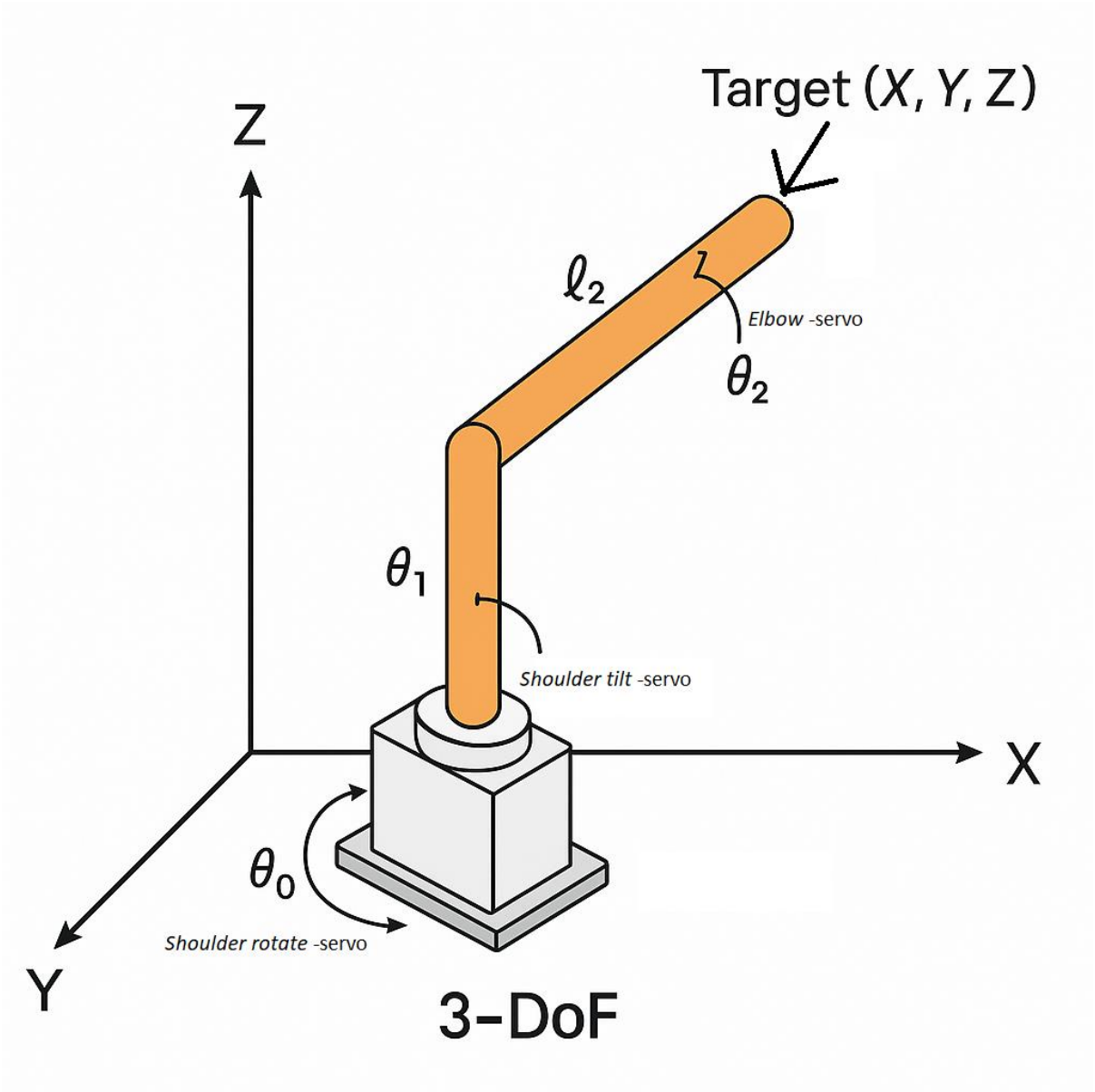

Figure 3. An illustration of the third dimension and servo implementation.

Coordinate calculation and solution

The extension to 3D space was implemented using the following main points:

- In the X,Y plane direction, the angle of *shoulder rotate* -servo was still used to turn the arm in the correct direction.
- After this, the 3D distance (projected to the X,Z plane) was converted to a 2D calculation, where the required angles were calculated for the *shoulder tilt* -servo and the *elbow* -servo.
- This required reprojecting the target coordinates in the upper arm and forearm directions.

In practice, the angle of *shoulder rotate* -servo was used to rotate the entire robot arm horizontally towards the target point. After this, *shoulder tilt* -servo and *elbow* -servo were given a projection from the target point to a 2D plane, where the previously built trigonometric calculation used it.

However, I was not very familiar with inverse kinematics, and understanding it required delving into geometry, trigonometry and their programmatic modelling. Often, problems did not appear immediately in the code, but only in the physical movement of the robot arm, which led to a continuous cycle of testing, finding errors and refining the calculation. Understanding gradually grew as theory and practice began to support each other. This stage was very educational: it further developed mathematical thinking, problem-solving skills and the ability to apply a theoretical model in a concrete system. Although the stage involved a lot of trial and error, the result was a working 3D inverse kinematics that created a solid foundation for precise and reliable motion control of the robotic arm.

Effects of physical structure

This stage also took into account the 3D configuration of the structure for the first time:

- *Shoulder rotate* -servo offset: servo offset: the servo's joint was not directly in the same position as the *shoulder tilt* -servo's joint, so a horizontal offset (L2) was calculated.
- This required additional transformations to the original coordinates before starting the trigonometric calculation.

This step made the robotic arm truly usable for 3D trajectories and laid the foundation for subsequent functions, such as wrist position control and gripper orientation. It also showed that the previous structures and software architecture could withstand the expansion without significant changes.

Modelling offsets and physical dimensions of the structure

Until now, inverse kinematics calculations were partly based on an ideal model, where the joints were located exactly in the same line, and the structure of the robot arm was assumed to be a complete continuation of the joints without structural deviations. In practice, however, the situation was different: for example, the servos were not placed directly on top of each other, but there were structural deviations between them and the parts had physical thicknesses that significantly affected the trajectories and kinematics.

In previous stages, these physical constraints had been partially considered, but not yet to the full extent. In this stage, a detailed structural modelling was carried out, which considered:

- The actual positions of the servo attachment point (for example, offset in the horizontal and vertical planes),
- The lengths of the parts between the joints (upper arm, forearm, clamps),
- Deviations from the ideal joint geometry caused by the frame structure.

Additionally, the physical angular limits (minimum and maximum) of each servo were specified. This was necessary to ensure that the inverse kinematics calculation did not produce positions that the servo could not reach – or that could cause mechanical collisions or loads.

Calibration and angle corrections

In practice, it was also observed that some of the servos did not fully follow the assumed angular orientation. For example, the *shoulder tilt* -servo physically pointed in a slightly different direction than the expected 90-degree position (not straight up but slightly tilted) (Figure 4.). This was due to limitations related to the robot arm's mountings. Such deviations were measured and compensated for programmatically using angle corrections, which were considered in the kinematics calculation before the actual joint angles were controlled for the servo.

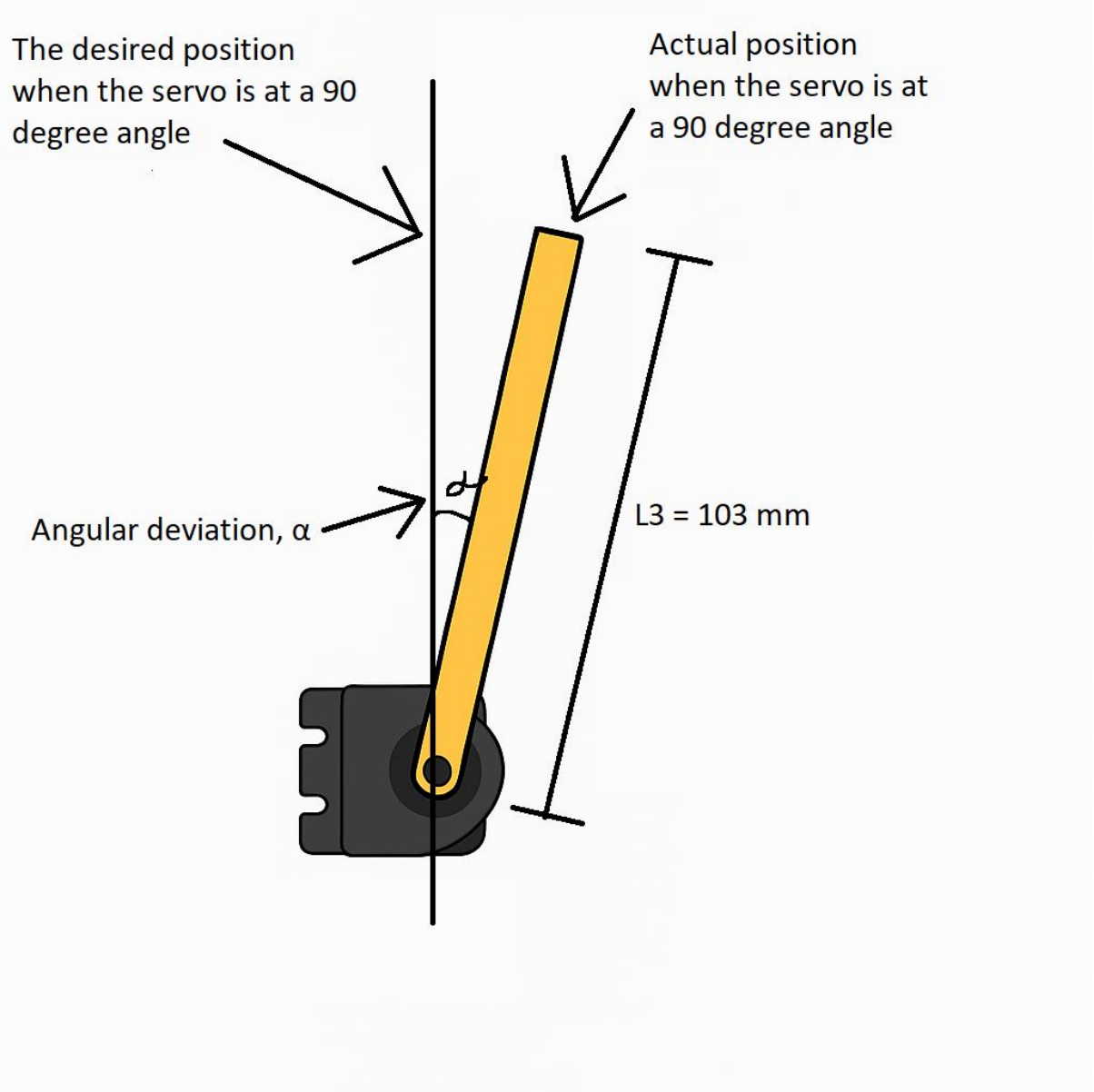

Figure 4. Illustration of angular deviation.

Movement validation

At the same time, a more detailed validation check was added to the program, ensuring that the given coordinates are achievable at all with the specified servo settings and structural constraints. This significantly improved the reliability of the arm and prevented situations where the program would try to control the robotic arm to a physically impossible position.

Wrist tilt control: 4-DoF (fourth degree of freedom)

Until now, IK (inverse kinematic) calculations only covered the position of the arm and the position of the gripper in 3D space. The next step was to also enable control of the wrist angle, so that the gripper head could be tilted in the desired direction. This added a 4-DoF (fourth degree of freedom) to the system and opened up the possibility of more versatile gripping movements and tool use (Figure 5.).

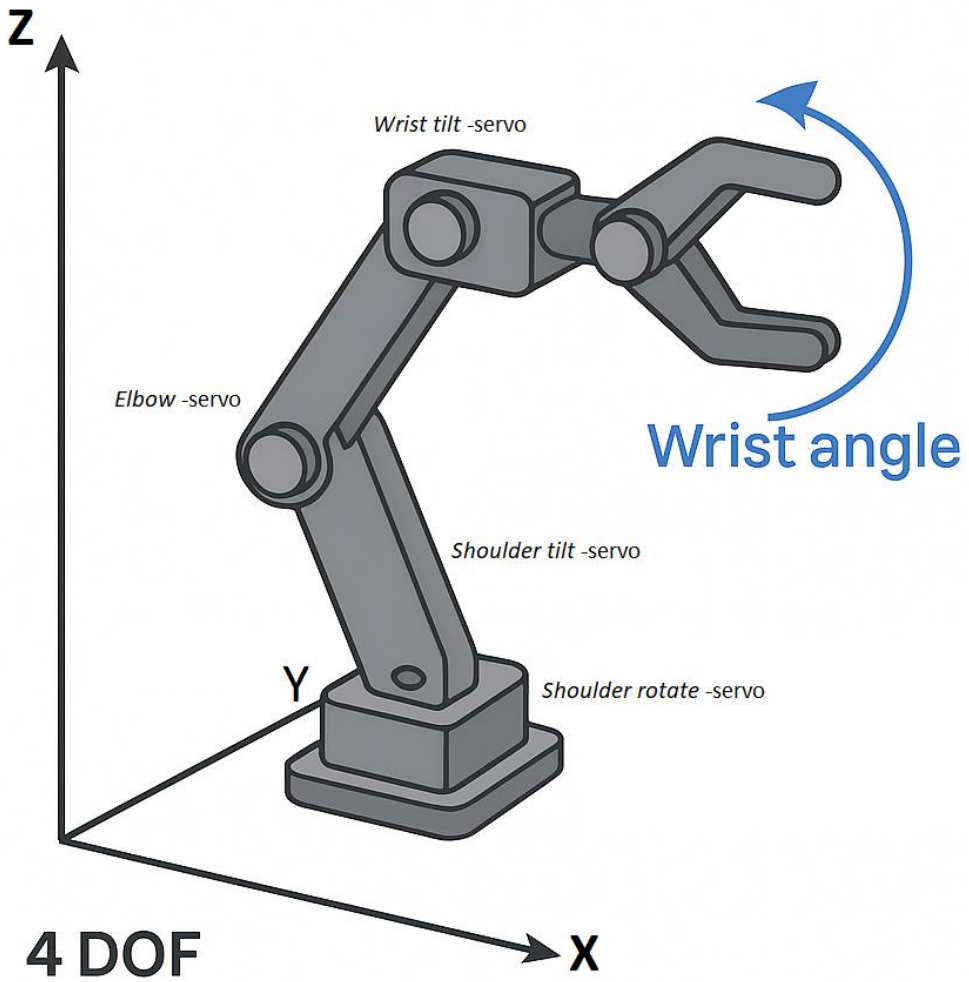

Figure 5. Illustration of wrist tilt.

Wrist tilt: Angle in global coordinate system

The goal was to be able to determine the wrist tilt angle relative to the global Z-axis:

- 90° meant that the grippers were horizontal.
- Values < 90° pointed downwards (e.g. in case of gripping from a tabletop).
- Values > 90° pointed upwards (e.g. in case of mounting upwards).

The servo controlling the wrist angle was commonly called the *wrist tilt* -servo.

Wrist angle calculation

The *wrist tilt* -servo was not an independent joint, but its position was determined by the chain formed by the upper arm and forearm. Therefore, the control of the wrist tilt servo could not be based on a simple absolute value, but its angle had to be calculated in relation to the reference angle formed by the position of the rest of the hand.

At this stage, the biggest challenge was to implement the logic where the user's desired wrist tilt angle (for example, 45°, i.e. diagonally downward) would be implemented regardless of the position of the rest of the robot arm. In other words, the desired wrist angle had to be expressed in a global coordinate system – not relative to the position of the forearm or upper arm, as in previous joints.

Achieving this required a new type of calculation, in which:

1. The reference angle formed by the forearm direction relative to the global coordinate system was calculated.
2. The desired wrist angle was compared to this.
3. The angle correction needed for the wrist was derived from this, so that the end result corresponds to the user's goal.

The implementation turned out to be surprisingly complex because it broke the previous chain logic, where each link was based on the previous corners. Now the corners had to be treated from a global perspective, which required both a new way of thinking and a new programmatic structure.

This phase was one of the most challenging of the entire project, but at the same time it provided a very concrete learning experience about how coordinate systems, angles and chains work in physical space. The final working solution was created through several tests, visualizations and experiments, and with it also came a deeper understanding of how complex motion can be controlled mathematically precisely – and still be simple for the user.

Practical implementation

Wrist tilt -servo was added to the existing servo control logic and its limits (min/max angles) were defined according to the servo settings. The calculation now returned four joint angles:

- ***Shoulder rotate***
- ***Shoulder tilt***
- ***Elbow***
- ***Wrist tilt***

The system still operated with the same command structure, but now in addition to coordinates, the desired wrist tilt angle could be specified separately.

This stage made the robotic arm more practical for a variety of tasks where the angle of approach to the part was critical. For example, picking objects from a box or accurately placing them on another component required precisely such fine-tuning.

Wrist and gripper control – moving to 6-DoF (six degrees of freedom)

Once the robot arm's shoulder and forearm trajectories and wrist tilt were under control, the project moved on to implementing the last two servos – the *wrist rotate* -servo (wrist rotation) and the gripper servo (opening and closing the grippers). With these, the robot arm's degrees of freedom increased to six, making it a true 6-DoF robot arm.

Wrist rotation

The *Wrist Rotate* -servo controls the rotational movement of the wrist around its own longitudinal axis, which means that it allows, for example, that the pliers can be rotated horizontally to a desired angle without having to change the other position of the robotic arm. This significantly increases the versatility of the arm in situations where an object needs to be handled in different positions or moved to another location with precise orientation.

To control wrist rotation, a new parameter, rotate, was added to the command system. It allowed the user to specify the direction of wrist rotation directly in the movement command, along with other position and attitude values. This change clearly demonstrated the modularity of the system – the new functionality could be smoothly extended without major changes to the existing code structure.

The wrist rotation is mechanically mounted as an extension of the previous joints and acts as an independent degree of freedom. The *wrist rotate* -servo is controlled directly to the desired angle and does not affect the calculation of the inverse kinematics of the other joints. This simplified the implementation, but at the same time allowed a significant increase in the usability of the arm.

Opening and closing the gripper (gripper servo)

The *gripper* -servo controls the opening and closing mechanism of the grippers. Unlike other servos, this is not related to position or angle adjustment but rather allows the gripping itself. Operation of the grippers was accomplished with simple commands such as GRIP OPEN and GRIP CLOSE, which turned the gripper servo to predetermined positions.

In the implementation, safe limits were defined for the gripper servo to prevent the grippers from accidentally opening or closing too forcefully. In addition, the servo was connected as part of the command interpretation so that the grippers can be easily controlled as part of a larger motion sequence.

Full 6-DoF (six-degrees of freedom) control

The introduction of these two servos completed the mechanical functionality of the robotic arm. A full 6-DoF (six degrees of freedom) enabled the robotic arm to move and manipulate its environment in terms of both position and orientation, and to grasp objects directly (Figure 6.). This opens the door to, for example, moving objects, sorting, or precision tasks in future development phases.

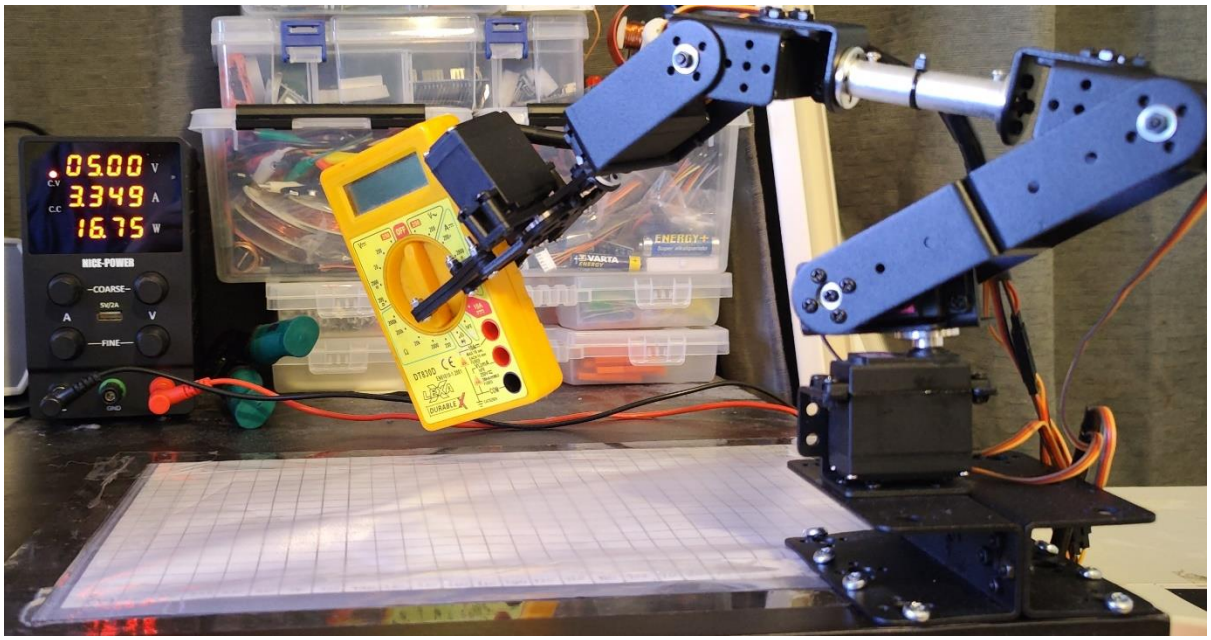

Figure 6. Demonstration of full six degrees of freedom (6-DoF) control by moving a piece.

Testing and iterative debugging

The final phase focused on overall system testing, error detection, and continuous fine-tuning. The goal of testing was to ensure that all servos move as expected, that given coordinates can be reliably reached, and that the robotic arm operates stably throughout its full range of motion.

Overall resting

The overall testing involved running various displacement commands and simulating realistic use cases such as lifting, moving and positioning an object. At the same time, the movement limits of each servo were examined, and individual angular movements were performed, ensuring that they remained within the specified minimum and maximum angles.

Iterative adjustment

During testing, several small errors were found, including:

- **Geometric deviations** – e.g. the physical angles of the servos did not fully correspond to the theoretical zero points.
- **The need for angle offsets** – for some servos, the zero angle meant a physically skewed position, which had to be compensated for programmatically.
- **Problems with edge areas** – in certain angle combinations, the arm did not reach the given point or behaved unstable.

These problems were addressed by fine-tuning the inverse kinematics calculation, servo controls, and setting safe limits. For example, the movement ranges were cut slightly so that the servos would not overload in the extreme areas or cause structural load on the mechanical parts.

Calibration

The testing also included servo-specific calibration: a precise zero angle was defined for each servo based on its physical structure, and these values were included in the inverse kinematics calculation. This ensured that the given angles actually corresponded to the expected position in reality.

Final result

Through testing and adjustments, the robotic arm was able to move precisely, safely, and predictably. This stage was crucial to making the system work in practice, not just theoretically. Iterative development also allowed for problems to be fixed quickly without the need for major structural changes.

Further development and future opportunities

Although the project achieved a functional and stable implementation of a six-degree-of-freedom (6-DoF) robotic arm, it offers ample opportunities for further development. One key development direction is trajectory planning, where the arm would move smoothly from point to point along an optimized path instead of moving directly at each angle. Another important area would be collision avoidance and environmental awareness, which could be approached using sensing – for example, ultrasonic or infrared sensors that detect obstacles around the arm. Also, building a feedback loop from, for example, servo load data or visual perception (e.g. via a camera) would enable more dynamic and intelligent control.

On the software side, it would be possible to develop a user-friendly graphical user interface through which the robotic arm could be controlled and programmed more easily. In addition, the robotic arm could be integrated into a larger automation system or utilized as a learning environment in artificial intelligence or machine vision projects. Overall, the project provides an excellent foundation for deepening expertise in combining robotics, mechanics, electronics, and programming into a real system.

Summary

This robotic arm project was a practical and versatile entity, combining mechanics, electronics and programming into a concrete and functional system. The goal of the project was to implement a 6-DoF (six-degree-of-freedom) robotic arm that could be controlled using inverse kinematics by giving it 3D coordinates. Along the way, the project was developed step by step from a simple 2D control to a full-fledged 6-DoF (six-degree-of-freedom) implementation, which also considered the physical properties of the robot structure, the angular limitations of the servos and the user interface for entering commands.

During the project, several challenges were encountered, such as angular deviations caused by the physical positions of the servos, limited space within the motion architecture, and computational constraints when moving from 2D to 3D. Solving these challenges required good problem-solving skills, and this project significantly developed systems thinking and the ability to analyse and implement complex entities. In particular, the theoretical understanding of inverse kinematics and its application to a practical robot structure provided valuable lessons from both a mathematical and software design perspective.

The project's relevance to working life is particularly evident in the fact that the problems to be solved, the technologies used, and the implementation method meet the requirements of modern automation, industrial robotics and embedded systems in many respects. Such skills are valuable, for example, in programming robots for production lines, automation design and research and development work.

In the future, the project could be further developed by adding, for example, trajectory planning, a visual user interface, environmental recognition and a sensor-based feedback loop. This would allow the robotic arm to operate autonomously in a dynamic environment and perform tasks with greater precision and intelligence. The project provided a strong foundation for deepening expertise in robotics and sparked ideas and enthusiasm for new potential applications.